Listing 7.3 Creating a custom analyzer

```
GET _analyze
{
  "tokenizer": "path_hierarchy",
  "filter": ["uppercase"],
  "text": "/Volumes/FILES/Dev"
}
```

This code uses a `path_hierarchy` tokenizer with an `uppercase` filter and produces three tokens from the given input text: "/VOLUMES", "/VOLUMES/FILES", and "/VOLUMES/FILES/DEV". The `path_hierarchy` tokenizer splits the text based on a path separator; so, three tokens tell us about the three folders in the hierarchy.

We've been talking about analyzer modules for the last few sections. Elasticsearch provides several of them; let's look at these built-in analyzers in detail.

7.3 Built-in analyzers

Elasticsearch provides eight out-of-the-box analyzers we can use in the text analysis phase. These analyzers usually suffice for basic cases, but if we need to create a custom one, we can do so by instantiating a new analyzer module with the required components. Table 7.3 lists the Elasticsearch analyzers.

Table 7.3 Built-in analyzers

Analyzer	Description
`standard`	The default analyzer that tokenizes input text based on grammar, punctuation, and whitespace. The output tokens are lowercased.
`simple`	Splits input text on any non-letters, such as whitespace, dashes, and numbers. Unlike the `standard` analyzer, the `simple` analyzer also lowercases the output tokens.
`stop`	A `simple` analyzer with English stop words enabled by default
`whitespace`	Tokenizes input text based on whitespace delimiters
`keyword`	Doesn't mutate the input text. The field's value is stored as is.
`language`	Helps work with human languages, as the name suggests. Elasticsearch provides dozens of analyzers for different language such as English, Spanish, French, Russian, Hindi, and so on.
`pattern`	Splits tokens based on a regular expression (regex). By default, all nonword characters help to split the sentence into tokens.
`fingerprint`	Sorts the tokens and removes duplicates to produce a single concatenated token

The `standard` analyzer is the default and is widely used during text analysis. Let's look at the `standard` analyzer with an example in the next section, after which we look at the other analyzers in turn.